

Python Mcqs:

1) Predict the output of following python programs:

```
r = lambda q: q * 2
s = lambda q: q * 3
x = 2
x = r(x)
x = s(x)
x = r(x)
print (x)
```

- a)34
 - b)42
 - c)24
 - d)1
-

2)

```
a = 4.5
b = 2
print (a//b)
```

- a)2
 - b)2.0
 - c)0
 - d)1
-

3)

```
) a = True
  b = False
  c = False

) if a or b and c:
    print ("GEEKSFORGEEKS")
) else:
    print ("geeksforgeeks")
```

- a)geeksforgeeks
 - b)GEEKSFORGEEKS
-

4)

```
a = True  
b = False  
c = False
```

```
if not a or b:  
    print (1)  
elif not a or not b and c:  
    print (2)  
elif not a or b or not b and a:  
    print (3)  
else:  
    print (4)
```

a)1

b)2

c)3

4)4

5)

```
count = 1
```

```
def doThis():  
  
    global count  
  
    for i in (1, 2, 3):  
        count += 1
```

```
doThis()
```

```
print (count)
```

a)2

b)3

c)4

d)1

6)

```
class Acc:
    def __init__(self, id):
        self.id = id
        id = 555
```

```
acc = Acc(111)
print (acc.id)
```

a)555

b)111

c)None of the above

d)1

7)

```
for i in range(2):
    print (i)

for i in range(4,6):
    print (i)
```

a)0 1 2 5

b)0\n 1\n2\n4\n5\n

c)0\n 1\n4\n5\n

d)0 1 4 5

8)

```
values = [1, 2, 3, 4]
numbers = set(values)
```

```
def checknums(num):
    if num in numbers:
        return True
    else:
        return False
```

```
for i in filter(checknums, values):
    print (i)
```

a)True\nTrue\nTrue\nTrue

b)1\n2\n3\n4\n

c)1 2 3 4

d){True,True,True,True}

9)

```
counter = {}

def addToCounter(country):
    if country in counter:
        counter[country] += 1
    else:
        counter[country] = 1

addToCounter('China')
addToCounter('Japan')
addToCounter('china')

print (len(counter))
```

a)3

b)2

c)0

d)4

10)

Predict the output of following Python Programs.

Program 1:

```
 class Geeks:
    def __init__(self, id):
        self.id = id

 manager = Geeks(100)
 manager.__dict__['life'] = 49

print (manager.life + len(manager.__dict__))
```

a)491

b)149

c)51

d)50

11)

```
a = "GeeksforGeeks "  
  
b = 13  
  
print (a + b)
```

a)GeeksforGeeks13

b)ASCII(GeeksforGeeks)+b

c) Error

d)(GeeksforGeeks)print for 13 times

12)

```
dictionary = {}  
dictionary[1] = 1  
dictionary['1'] = 2  
dictionary[1] += 1  
  
sum = 0  
for k in dictionary:  
    sum += dictionary[k]  
  
print (sum)
```

a)1

b)2

c)3

d)4

13)

```
dictionary = {1:'1', 2:'2', 3:'3'}  
del dictionary[1]  
dictionary[1] = '10'  
del dictionary[2]  
print (len(dictionary))
```

a)1

b)3

c)2

d)4

14)

```
nameList = ['Harsh', 'Pratik', 'Bob', 'Dhruv']
```

```
print nameList[1][-1]
```

a)Error

b)k

c)h

d)Pratik

15)

```
nameList = ['Harsh', 'Pratik', 'Bob', 'Dhruv']
```

```
pos = nameList.index("GeeksforGeeks")
```

```
print pos * 5
```

a)Error

b)Exception

c)-5

d)5

16)

```
geekCodes = [1, 2, 3, 4]
```

```
geekCodes.append([5,6,7,8])
```

```
print len(geekCodes)
```

```
print(geekCodes)
```

a)4

b)5

c)8

d)7

17)

```
def addToList(listcontainer):
```

```
    listcontainer += [10]
```

```
mylistContainer = [10, 20, 30, 40]
```

```
addToList(mylistContainer)
```

```
print len(mylistContainer)
```

a)5

b)4

c)1

d)6

18)

```
def gfgFunction():  
    "Geeksforgeeks is cool website for boosting up technical skills"  
    return 1  
  
print (gfgFunction.__doc__[17:21])
```

a)Error

b)cool

c) is

d)1

19)

```
class A(object):  
    val = 1  
  
class B(A):  
    pass  
  
class C(A):  
    pass  
  
print (A.val, B.val, C.val)  
B.val = 2  
print (A.val, B.val, C.val)  
A.val = 3  
print (A.val, B.val, C.val)
```

a) 111\n 121\n231

b) 111\n121\n323

c) 111\n323\n333

d) D)111\n 333\n221

20)

```
check1 = ['Learn', 'Quiz', 'Practice', 'Contribute']  
check2 = check1  
check3 = check1[:]  
  
check2[0] = 'Code'  
check3[1] = 'Mcq'  
  
count = 0  
for c in (check1, check2, check3):  
    if c[0] == 'Code':  
        count += 1  
    if c[1] == 'Mcq':  
        count += 10  
  
print (count)
```

a)2

b)12

c)10

d)4

21

```
def gfg(x,l=[]):  
    for i in range(x):  
        l.append(i*i)  
    print(l)
```

```
gfg(2)  
gfg(3,[3,2,1])  
gfg(3)
```

a)No matching arguments found

b)Error

c) [0, 1]

[3, 2, 1, 0, 1, 4]

[0, 1, 0, 1, 4]

d) [0,1,2,3,4,5]

[3,2,1]

[0,1,0,1,4]

22)

```
D = dict()  
for x in enumerate(range(2)):  
    D[x[0]] = x[1]  
    D[x[1]+7] = x[0]  
print(D)
```

a) KeyError

b) {0: 1, 7: 0, 1: 1, 8: 0}

c) {0: 0, 7: 0, 1: 1, 8: 1}

d) {1: 1, 7: 2, 0: 1, 8: 1}

23)

```
D = {1 : {'A' : {1 : "A"}, 2 : "B"}, 3 : "C", 'B' : "D", "D": 'E'}  
print(D[D[D[1][2]]], end = " ")  
print(D[D[1]["A"]][2])
```

- a) D C
 - b) E B
 - c) D B
 - d) E KeyError
-

24)

```
D = dict()  
for i in range (3):  
    for j in range(2):  
        D[i] = j  
print(D)
```

- a) {0: 0, 1: 0, 2: 0}
 - b) {0: 1, 1: 1, 2: 1}
 - c) {0: 0, 1: 0, 2: 0, 0: 1, 1: 1, 2: 1}
 - d) TypeError: Immutable object
-

25)

```
D = {1 : [1, 2, 3], 2: (4, 6, 8)}  
D[1].append(4)  
print(D[1], end = " ")  
L = [D[2]]  
L.append(10)  
D[2] = tuple(L)  
print(D[2])
```

- a) [1, 2, 3, 4] [4, 6, 8, 10]
 - b) [1, 2, 3, 4] ((4, 6, 8), 10)
 - c) '[1, 2, 3, 4] TypeError: tuples are immutable
 - d) [1, 2, 3, 4] (4, 6, 8, 10)
-

26)

```

import threading

barrier = threading.Barrier(4)

class thread(threading.Thread):
    def __init__(self, thread_ID, thread_name):
        threading.Thread.__init__(self)
        self.thread_ID = thread_ID
        self.thread_name = thread_name
    def run(self):
        print("ThreadID = " + str(self.thread_ID) + ", ThreadName = " +
self.thread_name + "\n")
        try:
            barrier = threading.Barrier(4)
            barrier.wait()
        except:
            print("barrier broken")
thread1 = thread(100, "GFG")
thread2 = thread(101, "Geeks")
thread3 = thread(102, "GeeksforGeeks")

thread1.start()
thread2.start()
thread3.start()

barrier.wait()

print("Exit")

```

- a) ThreadID = 100, ThreadName = GFG
ThreadID = 101, ThreadName = Geeks
ThreadID = 102, ThreadName = GeeksforGeeks
b) ThreadID = 100, ThreadName = GFG
ThreadID = 101, ThreadName = Geeks
ThreadID = 102, ThreadName = GeeksforGeeks
Exit
c) Compilation error
d) Runtime error

27)

```

import threading

class thread(threading.Thread):
    def __init__(self, thread_ID, thread_name):
        threading.Thread.__init__(self)
        self.thread_ID = thread_ID
        self.thread_name = thread_name
    def run(self):
        print(self.thread_name)

thread1 = thread(100, "GFG ")
thread2 = thread(101, "Geeks ")
thread3 = thread(102, "GeeksforGeeks ")

thread1.start()
thread2.start()
thread3.start()

print("Exit")

```

- a) GFG Geeks GeeksforGeeks Exit
 - b) Exit Geeks GeeksforGeeks GFG
 - c) GFG Exit GeeksforGeeks Geeks
 - d) None of the above
-

28)

```
import threading

class thread(threading.Thread):
    def __init__(self, thread_ID, thread_name):
        threading.Thread.__init__(self)
        self.thread_ID = thread_ID
        self.thread_name = thread_name
    def run(self):
        print(self.thread_name)

thread1 = thread(100, "GFG ")
thread2 = thread(101, "Geeks ")
thread3 = thread(102, "GeeksforGeeks ")

thread = []
thread.append(thread1)
thread.append(thread2)
thread.append(thread3)

thread1.start()
thread2.start()
for thread in thread:
    thread.join()

thread3.start()

print("Exit")
```

- a) GFG Geeks GeeksforGeeks Exit
 - b) Compilation error
 - c) Program will halt in between due to Runtime error
 - d) None of these
-

29)

```
import threading

i = 5

class thread(threading.Thread):
    def __init__(self, thread_ID, thread_name):
        threading.Thread.__init__(self)
        self.thread_ID = thread_ID
        self.thread_name = thread_name
    def run(self):
        i = i + 1
        print(i)

thread1 = thread(100, "GFG ")
thread2 = thread(101, "Geeks")

thread1.start()
thread2.start()
```

- a) 66
 - b) 67
 - c) Compilation error
 - d) Runtime error
-

30)

```
import threading

class thread(threading.Thread):
    def __init__(self, thread_ID):
        self.thread_ID = thread_ID
    def run(self):
        print(self.thread_ID)

thread1 = thread(100)

thread1.start()
```

- a) 100
 - b) Compilation error
 - c) Runtime error
 - d) None of these
-

31)

What is the output of the following program?

```
x = ['ab', 'cd']
for i in x:
    i.upper()
print(x)
```

Answer:_____

32)

```
x = ['ab', 'cd']
for i in x:
    x.append(i.upper())
print(x)
```

Ans:_____

33)

```
i = 1
while True:
    if i%3 == 0:
        break
    print(i)
    i += 1
```

Ans: _____

34)What is the Output?

```
x = 123
for i in x:
    print(i)
```

Ans: _____

35)

```
for i in [1, 2, 3, 4][::-1]:
    print (i)
```

Ans: _____

36)

```
i = 1
while True:
    if i % 007 == 0:
        break
    print(i)
    i += 1
```

1. 1 2 3 4 5 6.
 2. 1 2 3 4 5 6 7.
 3. error.
 4. None of these
-

37)

```
True = False
while True:
    print(True)
    break
```

1. False.
 2. True.
 3. Error.
 4. None of these
-

38)

```
i = 1
while True:
    if i % 3 == 0:
        break
    print(i)
    i += 1
```

1. 1 2 3.
 2. 1 2.
 3. Syntax Error.
 4. None of these
-

39)

```
str1 = '{2}, {1} and {0}'.format('a', 'b', 'c')
str2 = '{0}{1}{0}'.format('abra', 'cad')
print(str1, str2)
```

- a) c, b and a abracad0
 - b) a, b and c abracadabra
 - c) a, b and c abracadcad
 - d) c, b and a abracadabra
-

40)

```
a = 2
b = '3.77'
c = -8
str1 = '{0:.4f} {0:3d} {2} {1}'.format(a, b, c)
print(str1)
```

- a) 2.0000 2 -8 3.77
- b) 2 3.77 -8 3.77
- c) 2.000 3 -8 3.77
- d) 2.000 2 8 3.77

41)

```
import string

Line1 = "And Then There Were None"
Line2 = "Famous In Love"
Line3 = "Famous Were The Kol And Klaus"
Line4 = Line1 + Line2 + Line3
print("And" in Line4)
```

- a) True 2
 - b) True
 - c) False
 - d) False 2
-

42)

```
line = "I'll come by then."
eline = ""
for i in line:
    eline += chr(ord(i)+3)
print(eline)
```

- a) L*oo frph e| wkhq1
 - b) L*oo#frph#e|#wkhq1
 - c) l*oo@frph@e|\$wkhq1
 - d) O*oo#Frph#E|#wKhq1
-

43)

```
nums = [1, 2, 3, 4]
squared = [x ** 2 for x in nums]
print(squared)
```

- a) [1, 4, 9, 16]
 - b) [1, 2, 3, 4]
 - c) [2, 4, 6, 8]
 - d) [1, 8, 27, 64]
-

44) What is the purpose of the `__init__` method in a Python class?

- a) It is used to initialize class variables.
- b) It is a reserved keyword and cannot be used in user-defined classes.
- c) It is used to create a new instance of the class.

d) It is used to define the constructor for the class.

45)

```
x = 10
```

```
def foo():
```

```
    global x
```

```
    x += 5
```

```
foo()
```

```
print(x)
```

a) 10

b) 15

c) 5

d) 20

46) Which of the following is true about Python's Global Interpreter Lock (GIL)?

a) It allows only one thread to execute Python bytecode at a time in a single process.

b) It prevents the use of multi-threading in Python.

c) It is used to lock global variables in a multi-threaded environment.

d) It is used to prevent memory leaks in Python programs.

47)

```
line = "What will have so will"
L = line.split('a')
for i in L:
    print(i, end=' ')
```

a) ['What', 'will', 'have', 'so', 'will']

b) Wh t will h ve so will

c) What will have so will

d) ['Wh', 't will h', 've so will']

48)

```
from random import randrange
L = list()
for x in range(5):
    L.append(randrange(0, 100, 2)-10)

# Choose which of outputs below are valid for this code.
print(L)
```

a) [-8, 88, 8, 58, 0] b) [-8, 81, 18, 46, 0] c) [-7, 88, 8, 58, 0] d) [-8, 88, 94, 58, 0]

49)

```
from math import *
a = 2.13
b = 3.7777
c = -3.12
print(int(a), floor(b), ceil(c), fabs(c))
```

a) 2 3 -4 3 b) 2 3 -3 3.12 c) 2 4 -3 3 d) 2 3 -4 3.12

49)

```
import re
p = re.compile('\d+')
print(p.findall("I met him once at 11 A.M. on"))
p = re.compile('\d')
print(p.findall("I went to him at 11 A.M."))
```

a) ['11', '4', '1886', '11'] b) ['1141886'] ['1', '1'] c) ['11', '4', '1886'] ['11'] d) ['11', '4', '1886'] ['1', '1']

50)

```
import re
print(re.sub('ge', '**', 'Geeksforgeeks', fl))
print(re.sub('ge', '**', 'Geeksforgeeks'))
```

a) **eksfor**eks **eksfor**eks b) **eksfor**eks Geeksfor**eks c) **Geeksfor**geeks Geeksfor**geeks d) TypeError: 'str' object does not support item assignment

51)

```
import math
import random
L = [1, 2, 3000000000000000]
for x in range(3):
    L[x] = math.sqrt(L[x])

# random.choices() is available on Python 3.6.1 only.
string = random.choices(["apple", "carrot", "pineapple"], L, k = 1)
print(string)
```

a) ['pineapple'] b) ['apple'] c) 'pineapple' d) both a and b

52)

```
data = 50
try:
    data = data/0
except ZeroDivisionError:
    print('Cannot divide by 0 ', end = '')
else:
    print('Division successful ', end = '')

try:
    data = data/5
except:
    print('Inside except block ', end = '')
else:
    print('GFG', end = '')
```

- a) Cannot divide by 0 GFG
- b) Cannot divide by 0
- c) Cannot divide by 0 Inside except block GFG
- d) Cannot divide by 0 Inside except block

53)

```
data = 50
try:
    data = data/10
except ZeroDivisionError:
    print('Cannot divide by 0 ', end = '')
finally:
    print('GeeksforGeeks ', end = '')
else:
    print('Division successful ', end = '')
```

- a) Runtime error
 - b) Cannot divide by 0 GeeksforGeeks
 - c) GeeksforGeeks Division successful
 - d) GeeksforGeeks
-

54)

```
value = [1, 2, 3, 4]
data = 0
try:
    data = value[4]
except IndexError:
    print('GFG', end = '')
except:
    print('GeeksforGeeks ', end = '')
```

- a) GeeksforGeeks
- b) GFG
- c) GFG GeeksforGeeks
- d) Compilation error

• • •

55)

```
value = [1, 2, 3, 4]
data = 0
try:
    data = value[3]
except IndexError:
    print('GFG IndexError ', end = '')
except:
    print('GeeksforGeeks IndexError ', end = '')
finally:
    print('Geeks IndexError ', end = '')

data = 10
try:
    data = data/0
except ZeroDivisionError:
    print('GFG ZeroDivisionError ', end = '')
finally:
    print('Geeks ZeroDivisionError ')
```

- a) GFG ZeroDivisionError GFG ZeroDivisionError
- b) GFG ZeroDivisionError Geeks ZeroDivisionError
- c) Geeks IndexError GFG ZeroDivisionError Geeks ZeroDivisionError
- d) Geeks IndexError GFG ZeroDivisionError

56)

```
value = [1, 2, 3, 4, 5]
try:
    value = value[5]/0
except (IndexError, ZeroDivisionError):
    print('GeeksforGeeks ', end = '')
else:
    print('GFG ', end = '')
finally:
    print('Geeks ', end = '')
```

- a) Compilation error
 - b) Runtime error
 - c) GeeksforGeeks GFG Geeks
 - d) GeeksforGeeks Geeks
-

57)

```
print(bool('False'))
print(bool())
```

- 1. False, True
 - 2. None, None
 - 3. True, True
 - 4. True, False
-

58)

```
) print(not(4>3))
  print(not(5&5))
```

1. False, False
 2. None, None
 3. True, True
 4. True, False
-

59)

```
print(['love', 'python'][bool('gfg')])
```

1. love
 2. python
 3. gfg
 4. None
-

60)

```
mylist =[0, 5, 2, 0, 'gfg', '', []]  
print(list(filter(bool, mylist)))
```

1. [0, 0,]
 2. [0, 5, 2, 0, 'gfg', "", []]
 3. Error
 4. [5, 2, 'gfg']
-

61)

```
if (7 < 0) and (0 < -7):  
    print("abhi")  
elif (7 > 0) or False:  
    print("love")  
else:  
    print("geeksforgeeks")
```

1. geeksforgeeks
 2. love
 3. abhi
 4. Error
-

62)

```
tuple = (1, 2, 3, 4)
tuple.append( (5, 6, 7) )
print(len(my_tuple))
```

1. 1
 2. 2
 3. 5
 4. Error
-

63)

```
tuple = {}
tuple[(1,2,4)] = 8
tuple[(4,2,1)] = 10
tuple[(1,2)] = 12
_sum = 0
for k in tuple:
    _sum += tuple[k]
print(len(tuple) + _sum)
```

1. 34
 2. 12
 3. 31
 4. 33
-

64)

```
tuple1 = (1, 2, 4, 3)
tuple2 = (1, 2, 3, 4)
print(tuple1 < tuple2)
```

1. Error
 2. True
 3. False
 4. Unexpected
-

65)

```
tuple = (1, 2, 3)
print(2 * tuple)
```

1. (1, 2, 3, 1, 2, 3)
 2. (1, 2, 3, 4, 5, 6)
 3. (3, 6, 9)
 4. Error
-

66)

```
tuple=("Check")*3
print(tuple)
```

1. Unexpected
 2. (3Check)
 3. CheckCheckCheck
 4. Syntax Error
-

67) Tuples are_____ in python.

- a)Mutable
- b)Immutable
- c)data structure

d)both a&c

68) List is _____ in nature.

a)Mutable

b)Immutable

c)Cannot be determined

d)None of the above

69)

```
a = {'Hello':'World', 'First': 1}
b = {val: k for k, val in a.items()}
print(b)
```

{'Hello':'World', 'First': 1}

{'World': 'Hello', 1: 'First'}

Can be both A or B

None of the above

70)

Which of the following functions converts date to corresponding time in Python?

strptime()

strftime()

Both A and B

None of the above

71)

```
word = "Python Programming"
n = len(word)
word1 = word.upper()
word2 = word.lower()
converted_word = ""
for i in range(n):
    if i % 2 == 0:
        converted_word += word2[i]
    else:
        converted_word += word1[i]
print(converted_word)
```

pYtHoN PrOgRaMmInG

Python Programming

python programming

PYTHON PROGRAMMING

72)

```
a = "4, 5"
nums = a.split(',')
x, y = nums
int_prod = int(x) * int(y)
print(int_prod)
```

20

45

54

4,5

73)

```
square = lambda x: x ** 2
a = []
for i in range(5):
    a.append(square(i))

print(a)
```

[0, 1, 4, 9, 16]

[1, 4, 9, 16, 25]

[0, 1, 2, 3, 4]

[1, 2, 3, 4, 5]

74)

```
def tester(*argv):
    for arg in argv:
        print(arg, end = ' ')
tester('Sunday', 'Monday', 'Tuesday', 'Wednesday')
```

Sunday

Wednesday

Sunday Monday Tuesday Wednesday

None of the above.

75) As what datatype are the *args stored, when passed into a function?

List.

Tuple.

Dictionary.

None of the above.

76)

```
def tester(**kwargs):  
    for key, value in kwargs.items():  
        print(key, value, end = " ")  
tester(Sunday = 1, Monday = 2, Tuesday = 3, Wednesday = 4)
```

Sunday 1 Monday 2 Tuesday 3 Wednesday 4

Sunday 1

Wednesday 4

None of the above

77)

77) As what datatype are the *kwargs stored, when passed into a function?

Lists.

Tuples.

Dictionary.

None of the above

78) Which of the following blocks will always be executed whether an exception is encountered or not in a program?

try

except

finally

None of These

79)

```
set1 = {1, 3, 5}
set2 = {2, 4, 6}
print(len(set1 + set2))
```

3

6

0

Error

80) What keyword is used in Python to raise exceptions?

raise

try

goto

except

81)

```
s1 = {1, 2, 3, 4, 5}
s2 = {2, 4, 6}
print(s1 ^ s2)
```

{1, 2, 3, 4, 5}

{1, 3, 5, 6}

{2, 4}

None of the above

82) Which of the following is not a valid set operation in python?

Union

Intersection

Difference

None of the above

83)

```
a = [1, 2, 3, 4]
b = [3, 4, 5, 6]
c = [x for x in a if x not in b]
print(c)
```

[1, 2]

[5, 6]

[1, 2, 5, 6]

[3, 4]

84) Which of the following are valid escape sequences in Python?

`\n`

`\t`

`\\`

All of the above

85) **Which of the following are valid string manipulation functions in Python?**

`count()`

`upper()`

`strip()`

All of the above

86) Which of the following modules need to be imported to handle date time computations in Python?

`datetime`

`date`

`time`

`timedate`

87) How can assertions be disabled in Python?

Passing -O when running Python.

Assertions are disabled by default.

Both A and B are wrong.

Assertions cannot be disabled in Python.

88)

```
a = ([], "abc", [0], 1, 0)
print(list(filter(bool, a)))
```

['abc', [0], 1]

[1]

["abc"]

None of the above

89) In which language is python written?

C++

C

Java

None of these

90) What will be the result of the following expression in Python “2 ** 3 + 5 ** 2”?

65536

33

169

None of these

91)

```
s = {1, 2, 3, 3, 2, 4, 5, 5}
print(s)
```

{1, 2, 3, 3, 2, 4, 5, 5}

{1, 2, 3, 4, 5}

None

{1, 5}

92)

```
dict1 = {'first': 'sunday', 'second': 'monday'}
dict2 = {1: 3, 2: 4}
dict1.update(dict2)
print(dict1)
```

{'first': 'sunday', 'second': 'monday', 1: 3, 2: 4}

{'first': 'sunday', 'second': 'monday'}

{1: 3, 2: 4}

None of the above.

93)

```
def is_even(number):  
    message = f"{number} is an even number" if number % 2 == 0 else f"{number} is an odd number"  
    return message  
print(is_even(54))
```

54 is an even number

54 is an odd number

number is an even number

number is an odd number

94)

```
numbers = (4, 7, 19, 2, 89, 45, 72, 22)  
sorted_numbers = sorted(numbers)  
odd_numbers = [x for x in sorted_numbers if x % 2 != 0]  
print(odd_numbers)
```

[7, 19, 45, 89]

[2, 4, 22, 72]

[4, 7, 19, 2, 89, 45, 72, 22]

[2, 4, 7, 19, 22, 45, 72, 89]

95)

```
numbers = (4, 7, 19, 2, 89, 45, 72, 22)
sorted_numbers = sorted(numbers)
even = lambda a: a % 2 == 0
even_numbers = filter(even, sorted_numbers)
print(type(even_numbers))
```

filter

int

list

tuple

96)

What will be the type of the variable sorted_numbers in the below code snippet?

```
numbers = (4, 7, 19, 2, 89, 45, 72, 22)
sorted_numbers = sorted(numbers)
print(sorted_numbers)
```

List

Tuple

String

Int

97)

```
example = ["Sunday", "Monday", "Tuesday", "Wednesday"];
del example[2]
print(example)
```

['Sunday', 'Monday', 'Tuesday', 'Wednesday']

['Sunday', 'Monday', 'Wednesday']

['Monday', 'Tuesday', 'Wednesday']

['Sunday', 'Monday', 'Tuesday']

98)

```
a = [1, 2]  
print(a * 3)
```

Error

[1, 2]

[1, 2, 1, 2]

[1, 2, 1, 2, 1, 2]

99)

```
example = ["Sunday", "Monday", "Tuesday", "Wednesday"]  
print(example[-3:-1])
```

['Monday', 'Tuesday']

['Sunday', 'Monday']

['Tuesday', 'Wednesday']

['Wednesday', 'Monday']

100)

```
def check(a):  
    print("Even" if a % 2 == 0 else "Odd")  
  
check(12)
```

Even

Odd

Error

None

Answers:

1)c 2)b 3)b 4)c 5)c 6)b 7)c 8)b 9)a 10)c 11)c 12)d
13)c 14)b 15)b 16)b 17)a 18)b 19)b 20)b 21)c 22)c 23)d 24)b
25)b 26)a 27)d 28)c 29)d 30)c 36)1 37)3 38)3 39)d 40)a 41) b
42)b 43)a 44)d 45)b 46)a 47)b 48)a 49)b 49)d 50)b 51)d 52)a
53)a 54)b 55)c 56)d 57)4 58)1 59)2 60)4 61)2 62)4 63)4 64)3
65)1 66)3 67)d 68)a 69)b 70)a 71)a 72)a 73)a 74)c 75)b 76)a
77)c 78)c 79)d 80)a 81)b 82)d 83)a 84)d 85)d 86)a 87)a 88)a
89)b 90)b 91)b 92)a 93)a 94)a 95)a 96)a 97)b 98)d 99)a
100)a

31) ['ab', 'cd']

32)No Output

33)No Output

34)Error

35)Reverse of the list

